

# Workshop 2 — Machine Learning

## Unidad 2

### Pipelines, entrenamiento, comparación de modelos y validación cruzada — Regresión y Clasificación

#### Objetivo

Aplicar el ciclo completo de un proyecto de Machine Learning supervisado — limpieza y preprocesamiento de datos, partición correcta en train/validation/test, construcción de pipelines con ColumnTransformer, entrenamiento y comparación de múltiples algoritmos, y validación cruzada — sobre un problema de regresión y un problema de clasificación. Cada decisión metodológica (qué se limpia, cómo se transforma, qué se escala) debe estar justificada, no solo aplicada.

#### Instrucciones generales

- Trabajo en equipos de 3-4 personas.
- Cada equipo debe resolver AMBOS problemas: el de regresión (predicción de precios de vuelos) y el de clasificación (diagnóstico de enfermedad tiroidea), usando los dos datasets indicados abajo.
- El entregable son notebooks (Jupyter/Colab) con código y celdas de texto (markdown) explicando y justificando cada decisión — no basta con correr el código ni con reportar la métrica; hay que sustentar por qué se tomó cada decisión de limpieza, preprocesamiento y modelado.
- No se permite eliminar filas/columnas, ni elegir una técnica de encoding/escalamiento/imputación, sin justificar por qué esa era la opción apropiada para esa variable y esos algoritmos.
- Toda transformación que 'aprenda' de los datos (imputación, escalamiento, encoding) debe ajustarse (.fit()) únicamente sobre el conjunto de entrenamiento.

#### Bases de datos de trabajo

Regresión — Flight Price Prediction:

<https://www.kaggle.com/datasets/shubhambathwal/flight-price-prediction>

Clasificación — Thyroid Disease Data:

<https://www.kaggle.com/datasets/jainaru/thyroid-disease-data/data>

#### Fase 1 — EDA, limpieza y preprocesamiento (aplica a ambos datasets)

Repita este proceso completo tanto para el dataset de regresión como para el de clasificación. Cada punto requiere una celda de justificación, no solo la ejecución del código.

1. Lea la documentación de cada dataset y describa el problema de negocio/salud que representa, e identifique cuál es la variable objetivo (target) en cada caso.
2. Cargue el dataset con pandas, muestre head()/tail(), .shape y .info(). Construya el diccionario de datos (columna, tipo esperado, significado).
3. .describe() para las columnas numéricas. Interprete al menos 3 estadísticos por dataset en términos del problema.
4. Identifique valores nulos por columna, calcule su porcentaje, y proponga una estrategia de imputación o eliminación. Justifique la elección según el porcentaje de nulos y el tipo de variable (¿es razonable imputar con la media? ¿con la moda? ¿es mejor eliminar la fila o la columna?).
5. Identifique y elimine duplicados si existen. Reporte cuántos se eliminaron.
6. Revise inconsistencias de formato en columnas categóricas o de texto (mayúsculas/minúsculas, espacios, etiquetas equivalentes) y corrijalas.
7. Identifique outliers en al menos dos columnas numéricas relevantes usando el criterio IQR. Repórtelos y decida si se tratan o se documentan sin eliminar, justificando la decisión (¿tienen sentido de negocio, como vuelos de última hora muy caros, o son errores de captura?).
8. Genere las gráficas de EDA (barras, pie, histograma, scatter plot y boxplot) para las variables relevantes de cada dataset. Para cada gráfica incluya el código, la imagen y un párrafo de interpretación con una recomendación como responsable de la toma de decisiones.

9. Para cada variable categórica, decida y justifique la técnica de codificación apropiada (One-Hot, Ordinal o Binary Encoding), considerando si la variable tiene orden natural, su cardinalidad, y qué algoritmos se van a usar (recuerde que Label/Ordinal Encoding puede introducir un orden falso en modelos lineales o de distancia).
10. Decida y justifique si es necesario escalar las variables numéricas y con qué técnica (Min-Max, StandardScaler o RobustScaler), considerando la presencia de outliers detectada en el punto 7 y los algoritmos que va a entrenar (recuerde que KNN y la regresión son sensibles a la escala; los árboles no).

## Fase 2 — División de datos y Pipelines

11. Divida cada dataset en train / validation / test (ej. 70/15/15). Explique con sus palabras el propósito de cada uno de los tres conjuntos y por qué no basta con solo train/test.
12. Construya, para cada dataset, un Pipeline con ColumnTransformer que aplique el preprocesamiento numérico (imputación + escalamiento) y categórico (imputación + encoding) definidos en la Fase 1.
13. Verifique explícitamente en una celda de texto que su implementación NO tiene fuga de datos (data leakage): confirme que el .fit() de cada transformación se realizó solo sobre X\_train, y que sobre X\_val y X\_test únicamente se aplicó .transform().

## Fase 3 — Modelado: Regresión (Flight Price Prediction)

14. Entrene los siguientes modelos sobre el conjunto de entrenamiento para predecir el precio del vuelo: Regresión Lineal Múltiple, KNN Regressor, Decision Tree Regressor, Random Forest Regressor y Gradient Boosting Regressor.
15. Para cada modelo, genere las predicciones sobre el conjunto de validación.

## Fase 4 — Modelado: Clasificación (Thyroid Disease Data)

16. Entrene los siguientes modelos sobre el conjunto de entrenamiento para predecir el diagnóstico: KNN Classifier, Decision Tree Classifier, Random Forest Classifier y Gradient Boosting Classifier.
17. Para cada modelo, genere las predicciones sobre el conjunto de validación.

## Fase 5 — Métricas en train y validación: detección de overfitting/underfitting

18. Regresión: para cada uno de los 5 modelos, calcule MAE, MSE y  $R^2$  tanto sobre el conjunto de entrenamiento como sobre el de validación (dos evaluaciones por modelo). Compare el desempeño en train contra el de val para cada modelo: ¿hay señales de overfitting (muy buen desempeño en train pero notablemente peor en val) o de underfitting (desempeño pobre en ambos conjuntos)? Interprete cada métrica en el contexto de precios de vuelos (¿qué significa un MAE de X pesos/dólares? ¿qué tan bueno es el  $R^2$  obtenido?).
19. Clasificación: para cada uno de los 4 modelos, calcule Accuracy, Precision, Recall, F1-score y la matriz de confusión, también sobre train y sobre validación. Compare ambos conjuntos por modelo para identificar overfitting o underfitting. Interprete los resultados considerando que es un problema de diagnóstico médico: discuta qué es más costoso, un falso positivo o un falso negativo, y qué métrica debería priorizarse.
20. Construya una tabla comparativa que incluya train y validación para regresión (filas: modelos, columnas: MAE/MSE/ $R^2$  en train y en val) y otra igual para clasificación (Accuracy/Precision/Recall/F1 en train y en val).
21. Seleccione el mejor modelo de cada problema y justifique la elección — no se limite a señalar la métrica más alta en validación; discuta trade-offs (tiempo de entrenamiento, interpretabilidad, la señal de over/underfitting detectada en los puntos anteriores) apoyándose en lo visto en clase sobre cada algoritmo.

## Fase 6 — Evaluación final en el conjunto de Test

El conjunto de test debe permanecer sin tocar hasta este punto: es la única forma de obtener una estimación honesta de cómo se comportaría cada modelo con datos nuevos.

22. Evalúe TODOS los modelos entrenados (los 5 de regresión y los 4 de clasificación) sobre el conjunto de test, usando las mismas métricas calculadas en la Fase 5.
23. Construya una tabla comparativa final con los resultados en test para cada problema, y compárela contra la tabla de validación de la Fase 5: ¿el orden/ranking de los modelos se mantiene igual? ¿las métricas del mejor modelo en test son consistentes con las que obtuvo en validación, o hay una diferencia importante? Interprete qué significaría una diferencia grande entre val y test.

## Fase 7 — Cross Validation

24. Investigue y explique con sus propias palabras qué es la validación cruzada (K-Fold Cross Validation), cómo funciona y por qué puede dar una estimación más confiable del desempeño de un modelo que un único split de train/validation.
25. Implemente K-Fold Cross Validation (ej. `cross_val_score` o `cross_validate` de `sklearn`) sobre el mejor modelo encontrado para el problema de regresión, usando las métricas relevantes.
26. Implemente K-Fold Cross Validation sobre el mejor modelo encontrado para el problema de clasificación, usando las métricas relevantes.
27. Compare los resultados de cross validation contra el desempeño obtenido en el conjunto de validación original de la Fase 5: ¿el desempeño mejora, empeora o se mantiene similar? ¿La desviación estándar entre folds sugiere que el modelo es estable o sensible a qué datos recibe? Interprete.

## Fase 8 — Conclusiones

28. Redacte un párrafo de conclusiones generales comparando ambos problemas: ¿qué modelo funcionó mejor en cada caso y por qué, relacionándolo con las características de cada algoritmo vistas en clase (sesgo/varianza, sensibilidad a escala, manejo de categóricas, etc.)? ¿Qué limpieza fue más determinante? ¿Qué limitaciones tuvo el estudio y qué preguntas quedan abiertas? Incluya un subplot (2,2) con las gráficas que considere más importantes para sustentar sus hallazgos generales, para cada dataset.

## Entregable

Dos notebooks (.ipynb) por equipo uno para regresión y uno para clasificación, subidos a un repositorio de GitHub. Solo debe poner el link del repositorio en la plataforma.

**Nota:** Este workshop será sustentado por algunos estudiantes de forma aleatoria, estos estudiantes sustentarán en nombre de su equipo y su nota de sustentación será tomada en cuenta así:

$$(Sustentación + Entrega) / 2$$